RAG Foundry: A Framework for Enhancing LLMs for Retrieval Augmented Generation

Daniel Fleischer **Moshe Berchansky** **Moshe Wasserblat** **Peter Izsak**
Intel Labs
{daniel.fleischer, moshe.berchansky, moshe.wasserblat, peter.izsak}@intel.com

arXiv:2408.02545v1 [cs.CL] 5 Aug 2024

Abstract

Implementing Retrieval-Augmented Generation (RAG) systems is inherently complex, requiring deep understanding of data, use cases, and intricate design decisions. Additionally, evaluating these systems presents significant challenges, necessitating assessment of both retrieval accuracy and generative quality through a multi-faceted approach. We introduce RAG FOUNDRY, an open-source framework for augmenting large language models for RAG use cases. RAG FOUNDRY integrates data creation, training, inference and evaluation into a single workflow, facilitating the creation of data-augmented datasets for training and evaluating large language models in RAG settings. This integration enables rapid prototyping and experimentation with various RAG techniques, allowing users to easily generate datasets and train RAG models using internal or specialized knowledge sources. We demonstrate the framework effectiveness by augmenting and fine-tuning Llama-3 and Phi-3 models with diverse RAG configurations, showcasing consistent improvements across three knowledge-intensive datasets. Code is released as open-source in https://github.com/IntelLabs/RAGFoundry.

1 Introduction

Large Language Models (LLMs) have emerged as a transformative force in the field of AI, demonstrating an impressive ability to perform a wide range of tasks that traditionally required human intelligence (Brown et al., 2020; Kojima et al., 2022). Despite their impressive capabilities, LLMs have inherent limitations. These models can produce plausible-sounding but incorrect or nonsensical answers, struggle with factual accuracy, lack access to up-to-date information after their training cutoff and struggle in attending to relevant information in large contexts (Huang et al., 2023; Liu et al., 2023).

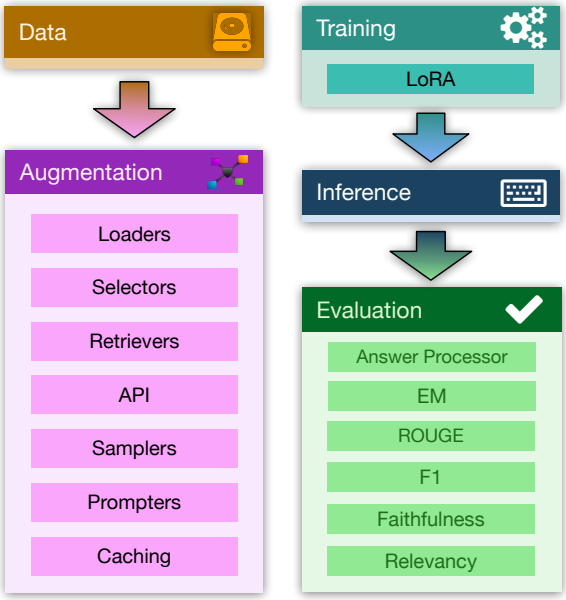

Figure 1: An overview of the RAG FOUNDRY framework: the **Data Augmentation** module persists RAG interactions into a dedicated dataset, which is then used for training, inference and evaluation.

Retrieval-Augmented Generation (RAG) enhances LLMs performance by integrating external information using retrieval mechanisms. Combining retrieval that leverages vast knowledge-bases outside the *knowledge* of the model, effectively addresses knowledge limitations, can reduce hallucinations, improve the relevance of generated content, provide interpretability and could be vastly more cost-efficient (Lewis et al., 2021; Mallen et al., 2022; Gao et al., 2023; Asai et al., 2023; Borgeaud et al., 2021; Peng et al., 2023; de Jong et al., 2023). Furthermore, recent research indicates that fine-tuning LLMs for RAG can achieve state-of-the-art performance, surpassing that of larger, proprietary models (Yu et al., 2024b; Liu et al., 2024).

However, the implementation of RAG systems is inherently complex and requires a series of intricate decisions that can significantly impact the performance of the system. This process de-

mands a thorough understanding of the data and use case, and often, solutions do not generalize well to other domains (Barnett et al., 2024; Balaguer et al., 2024). Some key RAG design decisions include text embedding, indexing parameters, retrieval algorithms, query building, and prompt design, among other considerations beyond the LLM configuration (Wang et al., 2024). Another issue is reproducibility: achieving consistent and comparable results across runs, datasets and tasks. Variations in training data, pre-processing steps, model configurations, and hardware can lead to discrepancies in performance, making it challenging for researchers and practitioners to replicate findings and build upon previous work. Additionally, evaluating RAG systems presents a challenge due to the dual reliance on retrieval accuracy and generative quality. These systems require a sophisticated evaluation suite that accounts for the interplay among the retrieved information, the formalization of data, and the generated output (Chen et al., 2023; Yu et al., 2024a; Es et al., 2024).

We introduce RAG FOUNDRY, an open-source python framework for developing sophisticated retrieval-augmented LLMs for RAG use-cases. The library supports researchers and practitioners in the nuanced task of enhancing the capabilities of LLMs in RAG use cases. It is highly customizable, facilitating rapid prototyping and experimentation across all aspects of RAG, including data selection, aggregation and filtering, retrieval, text processing, document ranking, few-shot generation, prompt design using templates, fine-tuning, inference, and evaluation. To cater to the specific needs of researchers, we designed the framework to function as an end-to-end experimentation environment. The backbone of the library consists of four distinct modules: data creation, training, inference, and evaluation. Each module is encapsulated and controlled by a configuration file, ensuring compatibility between the output of one module and the input of the next. This modular approach allows each step to be isolated and independently experimented with, enabling the production of multiple outputs and the concurrent execution of numerous experiments. Evaluation can be conducted on the generated outputs as well as on any feature within the data, including retrieval, ranking, and reasoning.

To illustrate the utility of the framework, we conducted experiments involving retrieval, fine-tuning, chain-of-thought (CoT) reasoning (Wu et al., 2023) and a negative distractor-documents technique (Zhang et al., 2024). We compared two widely accepted baseline models using various enhancement methods across three knowledge-intensive question-answering tasks, demonstrating the effectiveness of RAG FOUNDRY.

2 Related Work

There are numerous open-source tools related to the different aspects of RAG, namely inference, training and evaluation. LlamaIndex (Liu, 2022), LangChain (Chase, 2022) and Haystack (Pietsch et al., 2019) are well known libraries for composing RAG pipelines; however they are not focused on evaluation and their training capability is underdeveloped.

Hoshi et al. (2023) proposes a framework for developing RAG-based LLMs; while our processing may be similar in the sense of being comprised of custom individual steps, they do not introduce any form of training. Khattab et al. (2023, 2022) presents a different approach, where LLM prompting is represented as a programming language, to be optimized and compiled; a rather unique and general approach that could benefit RAG but has a high level of complexity due to the abstractions introduced. Saad-Falcon et al. (2024) focuses more on the evaluation aspect, by creating synthetic data and training an LLM critic to evaluate the RAG system. Hsia et al. (2024) studies aspects of retrieval on the performance of RAG; our RAG Foundry library is general and enables experimentation on all aspects of RAG: retrieval, text-processing, prompt design, model selection, inference and evaluations.

Recently, a concurrent work by Jin et al. (2024) proposes a RAG building framework, including some RAG implementations and datasets; we focus on extensibility, letting users define custom types of pipelines with custom components. Rau et al. (2024) presents a framework, sharing a similar design-principle of extensibility-through-configuration as ours; their library imposes a specific workflow structure (retriever, ranker, LLM) while our library is more general and does not imposes any specific paradigm.

3 RAG Foundry

The RAG FOUNDRY framework facilitates rapid prototyping and experimentation with various RAG settings and configurations. The library is composed of four modules: dataset creation, training,

```yaml
name: my_pipeline
cache: true
steps:
  - _target_: dataset_loaders.loaders.HFLoader
    inputs: main
    dataset_config:
      path: "Tevatron/wikipedia-trivia"
      split: train

  - _target_: dataset_loaders.loaders.LocalLoader
    inputs: fewshot-data
    filename: prepared-fewshot-data.jsonl

  - _target_: global_steps.sampling.ShuffleSelect
    inputs: main
    shuffle: 42
    limit: 10000

  - _target_:
  ↪  local_steps.retrievers.HaystackRetriever
    inputs: main
    pipeline_path: configs/qdrant.yaml
    query_key: query
    docs_key: positive_passages

  - _target_: global_steps.sampling.FewShot
    inputs: main
    input_dataset: fewshot-data
    k: 3
    output_key: fewshot_examples

  - _target_: local_steps.prompter.TextPrompter
    inputs: main
    prompt_file: prompts/basic.txt
    output_key: my_prompt
    mapping:
      question: query
      context: positive_passages
      fewshot: fewshot_examples
      answer: answers

  - _target_: global_steps.output.OutputData
    inputs: main
    file_name: TQA_train_processed.jsonl
```

Listing 1: Example of a dataset creation configuration. The example contains data loading, shuffling, sampling, retrieval, few-shot collection, prompt building and saving steps.

inference, and evaluation. Below, we expand on each of the modules and provide example configurations for running them.

3.1 Data Creation and Processing

The *processing* module facilitates the creation of context-enhanced datasets by persisting RAG interactions, which are essential for RAG-oriented training and inference (Berchansky et al., 2024; Liu et al., 2024; Yu et al., 2024b). These interactions encompass dataset loading, column normalization, data aggregation, information retrieval, template-based prompt creation, and various other forms of pre-processing. The processed data can be saved in a consistent, model-independent format, along with all associated metadata, ensuring compatibility and reproducibility across different models and experiments.

The processing module is comprised of an abstract pipeline with multiple steps, each defined by Python classes that implement specific data processing functionalities. These steps are categorized into two types:

- **Global Steps**: Can act on the dataset as a whole, making them useful for operations such as aggregations, group-by, examples filtering, join operations, and more.
- **Local Steps**: Operate on individual examples, making them suitable for tasks such as retrieval, text processing, and field manipulation.

The modular design allows for building flexible and efficient data processes, tailored to the needs of RAG-oriented training and inference. Steps can be categorized into the following non-exclusive categories:

- **Loaders**: Load datasets from the Hugging Face[1] hub or from local sources.
- **Selectors**: Filter examples, shuffle datasets, and select subset datasets.
- **Retrievers**: Integrate information from external databases, tools, libraries and pipelines.
- **Samplers**: Collect random examples or features from any dataset to compile few-shot or negative examples.
- **Prompters**: Format prompts using custom templates and keyword mappings.

The processing module supports the handling of **multiple datasets** at once, through global dataset sharing. This feature allows each step of the pipeline to access any of the loaded datasets, enhancing flexibility and allowing for complex processing procedures. Furthermore, the module includes **step caching**, which caches each pipeline step locally. This improves compute efficiency, and facilitates easy reproduction of results.

3.1.1 Example: Enhancing a Q&A Dataset

To showcase the effectiveness of the processing module, we demonstrate how to enrich a question-answering dataset with external informa-

[1] https://huggingface.co/

```yaml
model:
  _target_: ragfoundry.models.hf.HFTrain
  model_name_or_path:
  ↪  "microsoft/Phi-3-mini-128k-instruct"
  load_in_8bit: true
  lora:
    peft_type: "LORA"
    r: 16
    target_modules: ["qkv_proj"]
  completion_start: "<|assistant|>"

train:
  gradient_accumulation_steps: 4
  learning_rate: 2e-05
  lr_scheduler_type: "cosine"
  num_train_epochs: 1
  optim: "paged_adamw_8bit"

instruction: prompts/prompt_instructions/qa.txt
data_file: TQA_train_processed.jsonl
```

Listing 2: Example of a training configuration. Model and training parameters are specified, in addition to an instruction file containing the system prompt.

tion fetched using a retrieval pipeline, prepare few-shot examples and combine everything together using a prompt template. Listing 1 demonstrates how such a processing pipeline is defined using a YAML configuration. The main structure of the file is a list of steps, each defined by a `_target_` which points to the step implementation. Each step has `inputs`, which is a name or list of dataset names to act upon. Other keys in a step relate to specific step logic.

The first two steps in listing 1 load datasets from Hugging Face hub and from a local path. The third step shuffles and selects 10k examples from the main dataset. The forth step runs a Haystack-based (Pietsch et al., 2019) retrieval pipeline to retrieve relevant passages using questions from the loaded dataset as queries, storing them in `docs_key`. We note that different retrieval processes or frameworks (Liu, 2022; Chase, 2022; Lin et al., 2021) can be used in retrieval steps. The fifth step selects 3 few-shot examples from the secondary dataset, following a prompt generator step that loads a prompt template and replaces all given information according to the defined `mapping` dictionary. Lastly, the dataset is saved to a local path.

3.2 Training

We provide a *training* module to fine-tune models given the datasets created by the previous processing module. The training module relies on the well established training framework TRL[2] and supports advanced and efficient training techniques, e.g. LoRA (Hu et al., 2021). An example of a training configuration is presented in listing 2.

[2] https://github.com/huggingface/trl

```yaml
model:
  _target_: ragfoundry.models.hf.HFInference
  model_name_or_path:
  ↪  "microsoft/Phi-3-mini-128k-instruct"
  load_in_8bit: true
  instruction: prompts/prompt_instructions/qa.txt
  lora_path: /path/to/adapter
  generation:
    do_sample: false
    max_new_tokens: 50
    return_full_text: false

data_file: my-processed-data.jsnol
generated_file: model-predictions.jsonl
```

Listing 3: Example of an inference configuration. In addition to model and generation options, a system prompt can be defined.

3.3 Inference

The *inference* module generates predictions given the processed datasets created by the processing module. Inference is conceptually separated from the evaluation step, since it is more computationally demanding than evaluation. Additionally, one can run multiple evaluations on a single, prepared inference results file. An example configuration for generating predictions given a dataset is presented in listing 3.

3.4 Evaluation

The goal of the framework is augmenting LLMs for RAG. The *evaluation* module allows users to run collections of metrics to evaluate RAG techniques and tuning processes. The evaluation module loads the output of the inference module and runs a configurable list of metrics. Metrics are classes implemented in the library. These classes can be as simple as wrappers around other evaluation libraries, or can be implemented by the user. **Local metrics** can be run on individual examples, like Exact Match (EM), while **Global metrics** run on the entire dataset as a whole, e.g. Recall (for classification-based metrics). Metrics can use any field and metadata in the dataset, not just the input-output pairs. Some of the metrics implemented in the library include: a wrapper for the Hugging Face *evaluate* library, EM, F1, classification metrics, BERTScore (Zhang et al., 2019), Semantic Similarity and a wrapper for DeepEval[3] (for using

[3] https://github.com/confident-ai/deepeval

```yaml
answer_processor:
  _target_: ragfoundry.processing.RegexAnswer
  capture_pattern: "Answer: (.*)"
  stopping_pattern:

metrics:
  - _target_: ragfoundry.evaluation.HFEvaluate
    metric_names: ["rouge"]
  - _target_: ragfoundry.evaluation.EM
  - _target_: ragfoundry.evaluation.F1
  - _target_: ragfoundry.evaluation.BERTScore
    model: "microsoft/deberta-large-mnli"
  - _target_: ragfoundry.evaluation.Faithfulness
  - _target_: ragfoundry.evaluation.Relevancy
    embeddings: "BAAI/bge-small-en-v1.5"

results_file: my-evaluation.yaml
generated_file: model-prediction.jsonl
data_file: my-processed-data.jsonl
```

Listing 4: Example of an evaluation configuration; it contains an answer processor, as well as the list of metrics, with optional parameters, to run.

the RAGAS metrics (Es et al., 2024)). After the evaluation is completed, a results file is written to disk with the local and global metrics results.

Furthermore, the evaluation module uses a processing step called an **Answer Processor**, which can implement custom logic and serve many purposes, including cleaning and aligning outputs; for example, using regex, one can isolate answers, remove stop words, chain-of-thought reasoning, define a stopping criteria, process citations and attributions and any other form of processing needed for a given evaluation.

See listing 4 for a configuration example; it contains an answer processor that extracts an answer from an output, and a list of metrics to run.

4 Experiments: RAG Tuning

To illustrate the usage and usefulness of the RAG FOUNDRY library, we experiment with several possible RAG improvements to LLMs, and evaluate the results on three knowledge-intensive tasks.

4.1 RAG Augmentation Techniques

We explore several techniques for RAG augmentation, and use RAG FOUNDRY to easily implement and evaluate their benefit. As an initial step, we evaluate unmodified models; we set **Baseline** as a configuration that is defined by running unmodified models and without any external knowledge. We define a **RAG** setting that introduces top-relevant documents in a consistent prompt template format with a system instruction, and a **CoT** scheme which guides the model to use the retrieved context, explain the steps, quote relevant parts and produce a final answer. Complementing that, we explore fine-tuning recipes. We fine-tune the model in the **RAG** setup and denote is as **RAG-sft**. To complement **CoT**, we implemented a fine-tuning recipe, denoted as **CoT-sft**, introduced in (Zhang et al., 2024), where gold documents and purely distractor documents are used in the prompt, determined by probability, in conjunction with a CoT prompt. All prompt templates are included in appendix A.1.

4.2 Datasets

We evaluate our models on **TriviaQA** (Joshi et al., 2017), **PubmedQA** (Jin et al., 2019), and **ASQA** (Stelmakh et al., 2022) which are knowledge intensive question-answering datasets which benefit from external sources. The TriviaQA and PubmedQA datasets contain relevant context; for ASQA, retrieval was done over a Wikipedia corpus using a dense retriever[4]. Dataset sources and sizes are included in appendix A.2.

4.3 Models

We experiment with two representative models: Llama-3[5] (Touvron et al., 2023; AI@Meta, 2024) and Phi-3[6] (Abdin et al., 2024) as they represent robust capabilities and are ideal candidate models for RAG use case deployments.

4.4 Evaluation

We measure and report Exact Match (EM) for TriviaQA, STR-EM for ASQA, accuracy and F1 for PubmedQA. Additionally, we evaluate two RAGAS metrics (Es et al., 2024): Faithfulness and Relevancy. Faithfulness measures the relation between the generated text and the context. Relevancy measures the relation between the generated text and the query. These two metrics use the context as input for the LLM critic, so are only relevant in the RAG settings. The critic LLM used is GPT4-32k, version 0613. An embedder[7] is required for the relevancy evaluation.

4.5 Results

We present a comparative study of RAG augmentation techniques, on the TriviaQA, ASQA and PubmedQA datasets. Results are presented in table 1:

[4] BAAI/llm-embedder

[5] meta-llama/Meta-Llama-3-8B-Instruct.

[6] microsoft/Phi-3-mini-128k-instruct.

[7] BAAI/bge-small-en-v1.5.

Model	Method	TriviaQA			ASQA			PubmedQA			
		EM	Faith.	Rel.	STR-EM	Faith.	Rel.	Acc	F1	Faith.	Rel.
Phi-3 3.8B	Baseline	0.630	-	-	0.109	-	-	0.476	0.290	-	-
	RAG	0.876	0.821	0.836	0.294	0.685	0.895	0.530	0.281	-	-
	RAG-sft	0.878	0.777	0.750	0.252	0.717	0.833	**0.720**	**0.491**	-	-
	CoT	**0.923**	0.555	0.741	0.367	0.263	0.826	0.574	0.439	0.477	0.705
	CoT-sft	0.795	0.793	0.749	**0.386**	0.749	0.839	0.620	0.458	0.631	0.853
Llama-3 8B	Baseline	0.722	-	-	0.200	-	-	0.560	0.366	-	-
	RAG	0.828	0.783	0.746	0.285	0.610	0.861	0.556	0.398	-	-
	RAG-sft	**0.916**	0.704	0.714	0.291	0.653	0.854	**0.770**	**0.537**	-	-
	CoT	0.896	0.518	0.764	0.395	0.536	0.730	0.684	0.480	0.378	0.732
	CoT-sft	0.851	0.808	0.697	**0.422**	0.768	0.790	0.694	0.485	0.777	0.883

Table 1: Evaluation results of baseline and different RAG settings, for the three datasets and two models tested. In addition to the main metrics for each dataset, faithfulness and relevancy are reported for the relevant configurations. In bold are the best configurations per dataset, based on the main metrics.

main metrics for each dataset are displayed, as well as faithfulness and relevancy scores, as defined in (Es et al., 2024). For TriviaQA we observe the following: retrieved context improves the results, fine-tuning the RAG setting improves the results, fine-tuning on CoT reasoning (which includes training on a combination of gold passages and distractor passages) decreases performance. Best method is model dependent for this dataset. For ASQA, we similarly observe every method improves upon the baseline, CoT reasoning produces consistent improvement in both models, as well as fine-tuning of the CoT configuration, which shows to perform best. Finally, for PubmedQA, we observe that almost all methods improve upon the baseline (with one exception); CoT reasoning improves upon the untrained RAG setting, but upon fine-tuning, the RAG method appears to perform best in both models.

Inspecting the faithfulness and relevancy scores, notice that not all configurations are valid to be measured: these metrics require context, so are irrelevant for the baseline method. Additionally, in the PubmedQA dataset, the answers are binary Yes/No; only in the CoT configurations the LLMs produce a reasoning, which can be evaluated. Finally, the faithfulness and relevancy scores often do not correlate with the main metrics, neither with each other, possibly indicating they capture different aspects of the retrieval and generated results, and represent a trade-off in performance.

The results demonstrate the usefulness of RAG techniques for improving performance, as well as the need to carefully evaluate different aspects of a RAG system, on a diverse set of datasets, as effort on developing generalized techniques is ongoing.

5 Conclusion

We introduced RAG FOUNDRY, an open-source library dedicated to the task of RAG-augmentation of LLMs, namely fine-tuning LLMs to become better at RAG settings. The library is designed to serve as an end-to-end experimentation environment, enabling users to quickly prototype and experiment with different RAG techniques. We demonstrated the usefulness of the library by augmenting two models with RAG configurations, evaluating on three Q&A datasets and showing the benefit of RAG techniques, as well as of using multi-aspect metrics relevant for RAG systems evaluation.

Limitations and Future Plans

Our hope is that the library will be useful to as many people and use-cases as possible. However, due to time and resource constraint, we were able to demonstrate its usefulness on a subset of tasks and datasets. Future work can expand the evaluation to other tasks, as well as implementing other RAG techniques and evaluations.

Although we designed the library to be general and customizable, there might be specific workflows which will be difficult to run as-is and some code changes may be required. The library proved useful for our own research projects on a diverse set of datasets and tasks and extending it is easy and straightforward.

Finally, despite our best efforts to offer detailed documentation in the library, there could be some missing details regarding some functionality or specific use-cases. The code repository will accept suggestions, bug-fixes and pull requests.

Ethics Statement

In conducting our research we strive abiding to the highest ethical standards, including integrity, fairness, and societal benefit of our work. We prioritized data privacy and security throughout our research; any data used in our experiments was publicly available and did not contain any private information. We are committed to the principles of transparency and reproducibility; the methodologies, including data pre-processing, model training, and evaluation are documented in order to enable others to replicate our findings. Code is made available in an open repository. We advocate for the responsible use of LLMs and RAG augmentation. It is essential to exercise caution and verify the accuracy and reliability of generated text produced by LLMs. Hallucinations can have negative implications, and even when RAG methods can ameliorate some of these aspects, verification and inspections are needed.

References

Marah Abdin, Sam Ade Jacobs, Ammar Ahmad Awan, Jyoti Aneja, Ahmed Awadallah, Hany Awadalla, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Qin Cai, Martin Cai, Caio César Teodoro Mendes, Weizhu Chen, Vishrav Chaudhary, Dong Chen, Dongdong Chen, Yen-Chun Chen, Yi-Ling Chen, Parul Chopra, Xiyang Dai, Allie Del Giorno, Gustavo de Rosa, Matthew Dixon, Ronen Eldan, Victor Fragoso, Dan Iter, Mei Gao, Min Gao, Jianfeng Gao, Amit Garg, Abhishek Goswami, Suriya Gunasekar, Emman Haider, Junheng Hao, Russell J. Hewett, Jamie Huynh, Mojan Javaheripi, Xin Jin, Piero Kauffmann, Nikos Karampatziakis, Dongwoo Kim, Mahoud Khademi, Lev Kurilenko, James R. Lee, Yin Tat Lee, Yuanzhi Li, Yunsheng Li, Chen Liang, Lars Liden, Ce Liu, Mengchen Liu, Weishung Liu, Eric Lin, Zeqi Lin, Chong Luo, Piyush Madan, Matt Mazzola, Arindam Mitra, Hardik Modi, Anh Nguyen, Brandon Norick, Barun Patra, Daniel Perez-Becker, Thomas Portet, Reid Pryzant, Heyang Qin, Marko Radmilac, Corby Rosset, Sambudha Roy, Olatunji Ruwase, Olli Saarikivi, Amin Saied, Adil Salim, Michael Santacroce, Shital Shah, Ning Shang, Hiteshi Sharma, Swadheen Shukla, Xia Song, Masahiro Tanaka, Andrea Tupini, Xin Wang, Lijuan Wang, Chunyu Wang, Yu Wang, Rachel Ward, Guanhua Wang, Philipp Witte, Haiping Wu, Michael Wyatt, Bin Xiao, Can Xu, Jiahang Xu, Weijian Xu, Sonali Yadav, Fan Yang, Jianwei Yang, Ziyi Yang, Yifan Yang, Donghan Yu, Lu Yuan, Chengruidong Zhang, Cyril Zhang, Jianwen Zhang, Li Lyna Zhang, Yi Zhang, Yue Zhang, Yunan Zhang, and Xiren Zhou. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *Preprint*, arXiv:2404.14219.

AI@Meta. 2024. Llama 3 model card.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *Preprint*, arXiv:2310.11511.

Angels Balaguer, Vinamra Benara, Renato Luiz de Freitas Cunha, Roberto de M. Estevão Filho, Todd Hendry, Daniel Holstein, Jennifer Marsman, Nick Mecklenburg, Sara Malvar, Leonardo O. Nunes, Rafael Padilha, Morris Sharp, Bruno Silva, Swati Sharma, Vijay Aski, and Ranveer Chandra. 2024. RAG vs Fine-tuning: Pipelines, Tradeoffs, and a Case Study on Agriculture. *arXiv preprint*. ArXiv: 2401.08406 [cs].

Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. 2024. Seven failure points when engineering a retrieval augmented generation system. *Preprint*, arXiv:2401.05856.

Moshe Berchansky, Daniel Fleischer, Moshe Wasserblat, and Peter Izsak. 2024. Cotar: Chain-of-thought attribution reasoning with multi-level granularity. *Preprint*, arXiv:2404.10513.

Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George van den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego de Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, T. W. Hennigan, Saffron Huang, Lorenzo Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack W. Rae, Erich Elsen, and L. Sifre. 2021. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. *arXiv preprint*. ArXiv:2005.14165 [cs].

Harrison Chase. 2022. LangChain.

Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2023. Benchmarking Large Language Models in Retrieval-Augmented Generation. arXiv.

Michiel de Jong, Yury Zemlyanskiy, Nicholas FitzGerald, Joshua Ainslie, Sumit Sanghai, Fei Sha, and William Cohen. 2023. Pre-computed memory or

on-the-fly encoding? A hybrid approach to retrieval augmentation makes the most of your compute. Publisher: arXiv Version Number: 2.

Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta. Association for Computational Linguistics.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint*. ArXiv:2312.10997 [cs].

Yasuto Hoshi, Daisuke Miyashita, Youyang Ng, Kento Tatsuno, Yasuhiro Morioka, Osamu Torii, and Jun Deguchi. 2023. RaLLe: A Framework for Developing and Evaluating Retrieval-Augmented Large Language Models. *arXiv preprint*.

Jennifer Hsia, Afreen Shaikh, Zhiruo Wang, and Graham Neubig. 2024. RAGGED: Towards Informed Design of Retrieval Augmented Generation Systems. *arXiv preprint*. ArXiv:2403.09040 [cs].

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint*. ArXiv: 2106.09685 [cs].

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2023. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *arXiv preprint*. ArXiv:2311.05232 [cs].

Jiajie Jin, Yutao Zhu, Xinyu Yang, Chenghao Zhang, and Zhicheng Dou. 2024. FlashRAG: A Modular Toolkit for Efficient Retrieval-Augmented Generation Research.

Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William W. Cohen, and Xinghua Lu. 2019. PubMedQA: A Dataset for Biomedical Research Question Answering. *arXiv preprint*. ArXiv: 1909.06146 [cs, q-bio].

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. TriviaQA: A Large Scale Distantly Supervised Challenge Dataset for Reading Comprehension. *arXiv preprint*. ArXiv:1705.03551 [cs].

Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. 2022. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP. *arXiv preprint arXiv:2212.14024*.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2023. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.

Takeshi Kojima, S. Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. *ArXiv*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2021. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv preprint*.

Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. 2021. Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations. In *Proceedings of the 44th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2021)*, pages 2356–2362.

Jerry Liu. 2022. LlamaIndex.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *Preprint*, arXiv:2307.03172.

Zihan Liu, Wei Ping, Rajarshi Roy, Peng Xu, Chankyu Lee, Mohammad Shoeybi, and Bryan Catanzaro. 2024. ChatQA: Surpassing GPT-4 on Conversational QA and RAG. *arXiv preprint*. ArXiv: 2401.10225 [cs].

Alex Troy Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. 2022. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Annual Meeting of the Association for Computational Linguistics*.

Baolin Peng, Michel Galley, Pengcheng He, Hao Cheng, Yujia Xie, Yu Hu, Qiuyuan Huang, Lars Liden, Zhou Yu, Weizhu Chen, and Jianfeng Gao. 2023. Check Your Facts and Try Again: Improving Large Language Models with External Knowledge and Automated Feedback. Publisher: arXiv Version Number: 3.

Malte Pietsch, Timo Möller, Bogdan Kostic, Julian Risch, Massimiliano Pippi, Mayank Jobanputra, Sara Zanzottera, Silvano Cerza, Vladimir Blagojevic, Thomas Stadelmann, Tanay Soni, and Sebastian Lee. 2019. Haystack: the end-to-end NLP framework for pragmatic builders.

David Rau, Herv'e D'ejean, Nadezhda Chirkova, Thibault Formal, Shuai Wang, Vassilina Nikoulina, and S. Clinchant. 2024. BERGEN: A Benchmarking Library for Retrieval-Augmented Generation.

Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. *arXiv preprint*. ArXiv:2311.09476 [cs].

Ivan Stelmakh, Yi Luan, Bhuwan Dhingra, and Ming-Wei Chang. 2022. ASQA: Factoid Questions Meet Long-Form Answers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 8273–8288, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.

Xiaohua Wang, Zhenghua Wang, Xuan Gao, Feiran Zhang, Yixin Wu, Zhibo Xu, Tianyuan Shi, Zhengyuan Wang, Shizheng Li, Qi Qian, Ruicheng Yin, Changze Lv, Xiaoqing Zheng, and Xuanjing Huang. 2024. Searching for Best Practices in Retrieval-Augmented Generation. *arXiv preprint*.

Dingjun Wu, Jing Zhang, and Xinmei Huang. 2023. Chain of thought prompting elicits knowledge augmentation. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 6519–6534, Toronto, Canada. Association for Computational Linguistics.

Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2024a. Evaluation of Retrieval-Augmented Generation: A Survey. *arXiv preprint*. ArXiv:2405.07437 [cs].

Yue Yu, Wei Ping, Zihan Liu, Boxin Wang, Jiaxuan You, Chao Zhang, Mohammad Shoeybi, and Bryan Catanzaro. 2024b. RankRAG: Unifying Context Ranking with Retrieval-Augmented Generation in LLMs. *arXiv preprint*. ArXiv:2407.02485 [cs].

Tianjun Zhang, Shishir G. Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E. Gonzalez. 2024. Raft: Adapting language model to domain specific rag.

Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2019. BERTScore: Evaluating Text Generation with BERT. *ArXiv*.

A Implementation Details

A.1 Prompts

```
You are a helpful question answerer who can provide an answer given a question and relevant context.
```

Listing 5: System instruction used in the experiments.

```
Question: {query}
Context: {docs}
```

Listing 6: Template for inserting relevant documents as context.

```
Question: {query}
Context: {docs}

Answer this question using the information given in the context above. Here is things to pay attention to:
- First provide step-by-step reasoning on how to answer the question.
- In the reasoning, if you need to copy paste some sentences from the context, include them in
  ##begin_quote## and ##end_quote##. This would mean that things outside of ##begin_quote## and
  ##end_quote## are not directly copy paste from the context.
- End your response with final answer in the form <ANSWER>: $answer, the answer should be succinct.
```

Listing 7: Template for Chain-of-Thought reasoning.

A.2 Datasets

Datasets used:

- TriviaQA
- ASQA
- PubmedQA

Context size was $k = 5$, unless indicated otherwise.
Dataset sizes are:

Dataset	Training	Evaluation
TriviaQA	6000	1000
ASQA	4353	948
PubmedQA	10000	500

A.3 Training Details

Parameter	Value
LoRA r	16
LoRA α	16
LoRA Dropout	0.1
LoRA Bias	None
LoRA Modules	`qkv_proj`, Phi-3 `q/v_proj`, Llama-3
LR	1e-4
LR Scheduler	`cosine`
Warmup Ratio	0.03
Weight Decay	0.001
Batch Size	1
Epochs	1